



UNIVERSIDAD TÉCNICA PARTICULAR DE LOJA

La Universidad Católica de Loja

CARRERA DE COMPUTACIÓN

Programación Orientada a Objetos

Proyecto del Segundo Bimestre: StreamFlow

Nombres:

Pablo Andrés Ochoa Jaramillo

Pedro Sebastián Gómez Delgado

Profesor: Wayner Bustamante

Bimestre: 2

Fecha de entrega: 24/07/2026

StreamFlow

Informe técnico del proyecto

Catálogo de contenidos y suscripciones
Programación Orientada a Objetos - Segundo Bimestre - UTPL
Java + Ant + SQLite (JDBC) + JUnit 5
Arquitectura por capas (MVC) con principios SOLID

1. Qué es StreamFlow y qué hace

StreamFlow es una aplicación de consola que simula una plataforma de streaming. Administra un catálogo de contenidos (películas, series, documentales y podcasts) y usuarios con una suscripción. El costo que paga cada usuario depende de la calidad de sus contenidos favoritos (SD, HD o 4K), y además el sistema recomienda contenidos según los géneros que le gustan.

Funciones principales que ofrece el programa:

- Listar todos los contenidos del catálogo con su detalle y costo.
- Facturar a un usuario: calcular su costo mensual (suscripción base + costo de sus favoritos según la calidad).
- Recomendar contenidos a un usuario según los géneros de sus favoritos.

Los datos de contenidos y usuarios se guardan de forma permanente en una base de datos SQLite, de modo que persisten entre ejecuciones.

2. Flujo del usuario: qué pasa desde que entra

Cuando se ejecuta el programa, la clase Main actúa como "composición raíz": arma todas las piezas del sistema en orden y luego cede el control a la vista. El recorrido es el siguiente:

2.1. Arranque (clase Main)

1. Se crean las implementaciones de persistencia SQLite (ContenidoDaoSQLite y UsuarioDaoSQLite). Al construirse, ConexionBD.inicializar() crea las tablas "contenido" y "usuario" si aun no existen.
2. Se crean los servicios (lógica de negocio) inyectando los DAO por constructor.
3. Se crean los controladores inyectando los servicios.
4. Se crea la vista de consola (ConsolaVista) programada contra la interfaz IVista.
5. Se ejecuta sembrarDatos(): SOLO si la tabla de contenidos está vacía, inserta 4 contenidos de ejemplo (Interstellar, Breaking Bad, Planet Earth, Deep Questions) y un usuario "U1" con un favorito.
6. Main llama a vista.iniciar(...): a partir de aquí el control lo lleva la vista.

2.2. Menú principal (Consola Vista.iniciar)

La vista muestra un menú en bucle y lee la opción por teclado (Scanner). El menú es:

```
===== STREAMFLOW =====
1. Listar contenidos
2. Facturar usuario
3. Recomendar por género
0. Salir
=====
```

2.3. Qué hace cada opción (cadena de llamadas por capa)

Opción 1 - Listar contenidos:

Vista -> ContenidoControlador.listar() -> IContenidoService.listar()
-> IContenidoDao.listar() -> SELECT en SQLite -> List<Contenido>

La vista recibe la lista y muestra el detalle de cada contenido con obtenerDetalles() (cada tipo imprime sus campos propios y su costo).

Opción 2 - Facturar usuario:

Vista pide el ID -> UsuarioControlador.facturar(id)
-> IUserService.buscarPorId(id) (recupera el usuario)
-> ISuscripcionService.calcularCostoMensual(usuario)
= costo de la suscripción + suma de (costoBase x factor de calidad) de cada favorito

El resultado es el costo mensual total, que la vista muestra en pantalla. Si el ID no existe, devuelve 0.0.

Opción 3 - Recomendar por género:

Vista pide el ID -> UsuarioControlador.recomendar(id)
-> IUserService.buscarPorId(id)
-> IRecomendacionService.recomendarPorGenero(usuario)
= contenidos del mismo género que los favoritos, excluyendo los que ya son favoritos

La vista muestra la lista de recomendados (o un mensaje si no hay).

Opción 0 - Salir:

Se cierra el Scanner y termina el bucle; el programa finaliza.

3. Cómo se maneja MVC (arquitectura por capas)

El proyecto separa responsabilidades en capas, siguiendo el patrón Modelo-Vista-Controlador. Cada capa es un paquete y cada una habla SOLO con la capa inmediatamente inferior, y siempre a través de interfaces.

3.1. Cadena de dependencias

VISTA -> CONTROLADOR -> SERVICIO -> DAO -> SQLite
(consola) (orquestra) (lógica) (datos) (base de datos)

El paquete "modelo" es transversal: no depende de ninguna capa, y sus objetos (Contenido, Usuario, etc.) viajan por todas ellas como valores. El DAO los materializa desde la base de datos y suben hasta la vista como resultado.

3.2. Correspondencia con MVC

Rol MVC	Paquete(s)	Responsabilidad
Modelo	modelo, persistencia, servicio	Entidades del dominio, acceso a datos y lógica de negocio.
Vista	vista	Toda la entrada/salida por consola (menú, lectura de teclado, impresión).
Controlador	controlador	Intermediario: recibe la petición de la vista y coordina los servicios.

En sentido amplio, el "Modelo" de MVC agrupa aquí tres capas (entidades, persistencia y lógica), lo que permite que la vista no sepa nada de la base de datos ni de las reglas de negocio.

3.3. El controlador no es el "todo"

Un error común es pensar que el controlador conecta el modelo con todo. Aquí el controlador solo conoce las interfaces de servicio; no toca DAOs ni SQLite. Quien decide que implementación concreta recibe cada capa es Main (composición raíz). Esto es lo que permite, en las pruebas, inyectar un DAO en memoria en lugar del de SQLite.

4. Qué hace cada clase

4.1. Paquete modelo (entidades del dominio)

Clase	Que hace
Contenido (abstracta)	Clase base de todo contenido. Guarda id, título, género, calidad, duración y costo base. Define calcularCosto() = costoBase x factor de la calidad. Implementa las interfaces Detallable y Reproducible.
Pelicula	Subclase de Contenido. Añade "director". Implementa reproducir() y obtenerDetalles().
Serie	Subclase de Contenido. Añade "temporadas" y "episodios".
Documental	Subclase de Contenido. Añade "tema".
Podcast	Subclase de Contenido. Añade "presentador".
Calidad (enum)	Valores SD (factor 1.0), HD (1.5) y UHD_4K

	(2.0). El factor multiplica el costo base.
Género (enum)	Géneros posibles: ACCIÓN, COMEDIA, DRAMA, TERROR, DOCUMENTAL.
Usuario	Datos del usuario (id, nombre, email), su Suscripcion (composición) y su lista de favoritos (en memoria).
Suscripcion	Plan del usuario: id, fecha de inicio y costo mensual base.
contrato/Detallable	Interfaz con obtenerDetalles(): el objeto sabe describirse.
contrato/Reproducible	Interfaz con reproducir(): el objeto sabe reproducirse.

4.2. Paquete persistencia (acceso a datos)

Clase	Que hace
IContenidoDao / IUsuarioDao	Interfaces con el CRUD: insertar, listar, buscarPorId, actualizar, eliminar.
ContenidoDaoSQLite	Guarda contenidos en SQLite en una tabla única "contenido" con una columna "tipo" y columnas "extra1"/"extra2" para los campos propios de cada subclase. Traduce objeto <-> fila.
UsuarioDaoSQLite	Guarda usuarios en SQLite; la suscripción se guarda en columnas planas de la misma fila. Los favoritos no se persisten.
ContenidoDaoMemoria / UsuarioDaoMemoria	Implementaciones en memoria (con una lista) usadas en las pruebas unitarias, para no depender de la base de datos real.
ConexionBD	Clase de utilidad: abre la conexión a SQLite (con PRAGMA foreign_keys = ON) y crea las tablas si no existen.

4.3. Paquete servicio (lógica de negocio)

Clase	Que hace
IContenidoService / ...Impl	Registrar (validando id y duplicados) y listar contenidos.
IUsuarioService / ...Impl	Registrar y buscar usuarios por id.
ISuscripcionService / ...Impl	calcularCostoMensual(usuario): suma el costo de la suscripción más el costo de cada favorito (recalculado desde el DAO para respetar cambios de precio).
IRecomendacionService / ...Impl	recomendarPorGenero(usuario): devuelve contenidos del mismo género que los favoritos, sin incluir los que ya son favoritos.

4.4. Paquete controlador (coordinación)

Clase	Que hace
ContenidoControlador	Expone registrar() y listar() a la vista, delegando en IContenidoService.
UsuarioControlador	Expone registrar(), facturar(id) y recomendar(id); orquesta los servicios de usuario, suscripción y recomendación.

4.5. Paquete vista y arranque

Clase	Que hace
IVista	Interfaz de la vista: mostrarMenu(), mostrarContenidos(...) e iniciar(...).
ConsolaVista	Implementación por consola: dibuja el menú, lee opciones con Scanner en un bucle y muestra resultados y mensajes.
Main	Punto de entrada. Composición raíz: crea y conecta DAOs, servicios, controladores y vista, siembra datos de ejemplo y arranca la vista.

5. Cómo se manejan los principios SOLID

S - Responsabilidad única (SRP)

Cada clase tiene una sola razón para cambiar. La vista solo hace entrada/salida, el DAO solo persiste, el servicio solo aplica reglas de negocio y el controlador solo coordina. Un ejemplo claro: el bucle del menú (Scanner) se movió de Main a ConsolaVista, para que Main quede únicamente como composición raíz y la interacción viva en la vista.

O - Abierto/Cerrado (OCP)

El sistema está abierto a extensión y cerrado a modificación. Para agregar un nuevo tipo de contenido basta crear una subclase de Contenido (como se hizo con Pelicula, Serie, Documental y Podcast) sin modificar las clases existentes. Igualmente, cambiar el precio del 4K es cambiar el factor en el enum Calidad, sin tocar la lógica de cálculo.

L - Sustitución de Liskov (LSP)

Cualquier subclase de Contenido puede usarse donde se espera un Contenido sin romper el programa. La vista, los servicios y los DAO trabajan con el tipo Contenido y funcionan igual con una Pelicula, una Serie, un Documental o un Podcast, porque todas respetan el contrato de la clase base (calcularCosto, reproducir, obtenerDetalles).

I - Segregación de interfaces (ISP)

Las interfaces son pequeñas y específicas, no "gordas". Detallable y Reproducible están separadas (una clase implementa solo lo que necesita). La lógica de negocio se divide en cuatro interfaces

de servicio (contenido, usuario, suscripción y recomendación) en lugar de una sola con todos los métodos, y la persistencia tiene una interfaz DAO por entidad.

D - Inversión de dependencias (DIP)

Las capas dependen de abstracciones, no de implementaciones concretas. Los servicios dependen de IContenidoDao / IUserDao (no de las clases SQLite), y los controladores dependen de las interfaces de servicio. La única clase que conoce las implementaciones concretas es Main, que las inyecta por constructor. Gracias a esto, en las pruebas se inyecta un DAO en memoria en lugar del de SQLite sin cambiar una sola línea de los servicios.

```
// En producción (Main):      new ContenidoServiceImpl(new ContenidoDaoSQLite())
// En pruebas (test):         new ContenidoServiceImpl(new ContenidoDaoMemoria())
```

6. Pruebas unitarias (JUnit 5)

El proyecto incluye pruebas automatizadas con JUnit 5 que verifican el CRUD de los DAO en memoria y la lógica de los servicios (por ejemplo, que el costo mensual aplique correctamente el factor de calidad). Las pruebas usan los DAO en memoria como "dobles" para no depender de la base de datos, lo que solo es posible gracias a la inversión de dependencias (DIP) descrita arriba.

```
...
BAT:
cd: C:\Users\pablo\Documents\UTPL\SEGUNDO CICLO\PROGRAMAS\SEGUNDO BIMESTRE 2\StrawFlowProyectoBimestral2\build\build-jar.pr
...
Updating property file: C:\Users\pablo\Documents\UTPL\SEGUNDO CICLO\PROGRAMAS\SEGUNDO BIMESTRE 2\StrawFlowProyectoBimestral2\build
...
// ContenidoDaoMemoriaTest
// listarDevuelveCopiaIndependiente() ?
// eliminarExistenteLoQuita() ?
// insertarYBuscarPorIdDevuelveElContenido() ?
// insertarNullDevuelveFalse() ?
// actualizarReemplazaElContenido() ?
// buscarPorIdInexistenteDevuelveNull() ?
// insertarIdDuplicadoDevuelveFalse() ?
// eliminarInexistenteDevuelveFalse() ?
// actualizarInexistenteDevuelveFalse() ?
// ContenidoServiceImplTest
// registrarContenidoValidoDevuelveTrue() ?
// registrarIdDuplicadoDevuelveFalse() ?
// listarSinRegistrosDevuelveListaVacía() ?
// registrarNullDevuelveFalse() ?
// registrarIdVacioDevuelveFalse() ?
// RecomendacionServiceImplTest
// noRecomiendaContenidoDeOtroGenero() ?
// noRecomiendaLosContenidosQueYaSonFavoritos() ?
// usuarioSinFavoritosDevuelveListaVacía() ?
// usuarioNullDevuelveListaVacía() ?
// recomiendaOtroContenidoDelMismoGenero() ?
// ContenidoPolimorfismoTest
// losDetallesIncluyenElAtributoEspecificoDeCadaTipo() ?
// calcularCostoEsHeredadoYseComportaIgualEnTodaLaJerarquia() ?
// cadaSubclaseDevuelveSusPropiosDetalles() ?
// todoContenidoEsReproducibleYdetallable() ?
// todasLasSubclasesSeProcesanComoContenido() ?
// elCostoNoDependeDelTipoConcretoSinoDelEstado() ?
// reproducirImprimeUnMensajeDistintoSegunElTipo() ?
// SuscripcionServiceImplTest
// useElPrecioActualDelDaoNoElDelFavoritoEnMemoria() ?
// usuarioNullDevuelveCero() ?
// aplicaElFactorDeCalidadDeLosFavoritos() ?
// sinFavoritosDevuelveSoloElCostoDeLaSuscripcion() ?
// sumaVariosFavoritosConDistintaCalidad() ?
// UsuarioServiceImplTest
// agregarFavoritoSoloFuncionaConUsuarioYcontenidoValidos() ?
// buscarPorIdExistenteDevuelveElUsuario() ?
// registrarIdDuplicadoDevuelveFalse() ?
// registrarUsuarioValidoDevuelveTrue() ?
// buscarPorIdNuloOvacioDevuelveNull() ?
// registrarNullDevuelveFalse() ?
...

// ContenidoDaoSQLiteTest
// elTipoConcretoSeConservaAlLeerDeLaBase() ?
// listarIncluyeLosContenidosInsertados() ?
// losAtributosPropiosDeCadaSubclaseSePersisten() ?
// buscarPorIdInexistenteDevuelveNull() ?
// losDatosPersistenParaUnaInstanciaNuevaDelDao() ?
// actualizarCambiaLosDatosEnLaBase() ?
// insertarIdDuplicadoDevuelveFalse() ?
// insertarGuardaElContenidoEnLaBase() ?
// eliminarQuitaElContenidoDeLaBase() ?
// eliminarInexistenteDevuelveFalse() ?
// actualizarInexistenteDevuelveFalse() ?
// elContenidoLeidoConservaTodosSusDatos() ?
// JUnit Vintage
Test run finished after 698 ms
[ 11 containers found ]
[ 0 containers skipped ]
[ 11 containers started ]
[ 0 containers aborted ]
[ 11 containers successful ]
[ 0 containers failed ]
[ 61 tests found ]
[ 0 tests skipped ]
[ 61 tests started ]
[ 0 tests aborted ]
[ 61 tests successful ]
[ 0 tests failed ]

test:
BUILD SUCCESSFUL (total time: 1 second)
```

7. Cómo configurar JUnit 5 en un proyecto Ant de NetBeans

A diferencia de Maven, Ant no descarga ni gestiona dependencias por su cuenta: cada librería hay que bajarla a mano y enlazarla al proyecto. Para usar JUnit 5 en un proyecto Ant de NetBeans el proceso es el siguiente.

7.1. Descargar las librerías (JAR) de JUnit 5

Se descarga el JAR consolidado de JUnit 5. La opción más sencilla es junit-platform-console-standalone (en este proyecto se usó la versión 1.11.3) desde el repositorio central de Maven. Un solo archivo trae dentro todo lo necesario: la API Jupiter, su motor de ejecución, el lanzador de la plataforma, opentest4j y apiguardian; así se evita descargar cinco JAR sueltos.

El archivo se guarda en una carpeta dentro del proyecto, por convención lib.

7.2. Agregar la librería al proyecto en NetBeans

7. Abrir el proyecto Ant en NetBeans.
8. En la pestaña Projects, clic derecho sobre el nombre del proyecto y elegir Properties.
9. Seleccionar la categoría Libraries en el panel izquierdo.
10. Entrar a la pestaña de librerías de prueba (Test Libraries).
11. Pulsar Add JAR/Folder.
12. Buscar y seleccionar el archivo junit-platform-console-standalone.jar descargado.
13. Aceptar con OK para guardar los cambios.

El JAR se enlaza únicamente al classpath de pruebas (javac.test.classpath en nbproject/project.properties), nunca al de producción: el programa compilado no depende de JUnit, solo las pruebas.

7.3. Crear y ejecutar las pruebas

14. Abrir la clase que se desea probar.
15. Pulsar Alt + Insert (o clic derecho sobre el código) y elegir Create/Update Tests.
16. Elegir JUnit 5 en el desplegable de frameworks y aceptar con OK.
17. Escribir las pruebas con las anotaciones estándar de JUnit 5: @Test para cada método de prueba y Assertions.assertEquals() (o los demás asserts) para verificar resultados.
18. Para ejecutar, clic derecho sobre el archivo de prueba y elegir Test File, o pulsar Ctrl + F6.

7.4. Ajuste necesario en este proyecto

Con la configuración anterior las pruebas ya compilan, pero al ejecutarlas desde el IDE apareció un problema propio de Ant: la tarea <junit> que trae Ant es de JUnit 4, no de JUnit 5. Al lanzar las pruebas informaba errores del tipo "Invalid test class ... is not public / No runnable methods" en todas las clases, y aun así terminaba con BUILD SUCCESSFUL porque el proyecto ignora las pruebas fallidas por defecto.

La solución fue añadir al build.xml un target propio (test-junit5) que ejecuta el lanzador de consola de JUnit 5 en lugar de la tarea antigua, y sobrescribir el target test para que el botón de pruebas del IDE use ese lanzador. Con ese ajuste las 61 pruebas del proyecto se ejecutan y pasan correctamente.